

Un **sistema distribuido** es un conjunto de computadoras independientes que se comunican por una red y que, para el usuario, funcionan como un solo sistema. WhatsApp, la banca móvil o el Moodle de la universidad son sistemas distribuidos: detrás de cada uno hay muchos servidores conversando entre sí.

Tres ideas que conviene tener presentes desde el primer día:

- **Cada pieza corre por separado.** Llamamos *nodo* a cada una.
- **Se comunican solo por la red.** No comparten memoria: si un nodo quiere algo de otro, le envía un mensaje.
- **Las piezas fallan por separado.** Un nodo puede caerse mientras los demás siguen. A eso le llamamos *falla parcial*.

En esta clase armamos el sistema más pequeño posible que cumple esas tres ideas: una aplicación web hecha con Laravel y una base de datos PostgreSQL, cada una en su propio nodo. Para crear esos nodos en nuestro computador usamos **Docker**: cada nodo es un *contenedor*, una especie de computadora virtual liviana.

Cómo se ve nuestro sistema

El navegador solo entra por app; app encuentra a db por su nombre



El navegador pide la página por HTTP al nodo app, la única puerta abierta hacia Windows. Para armar la respuesta, app consulta al nodo db por la red interna, llamándolo por su nombre.

Conceptos

Imagen y contenedor. Una *imagen* es una plantilla: describe una máquina con todo lo que necesita (sistema operativo, PHP, librerías). Un *contenedor* es una máquina en funcionamiento creada a partir de esa imagen.

Dockerfile. Es la receta para construir una imagen propia. Sus instrucciones (FROM, RUN, WORKDIR) son estándar de Docker; lo que va después de cada una lo decidimos según lo que necesita nuestra aplicación. Cada línea existe porque resuelve una necesidad concreta. Escribirlo evita el clásico "en mi



máquina sí funciona": con la misma receta, cualquier computador construye exactamente la misma máquina.

Docker Compose. Un sistema distribuido tiene varios nodos que deben arrancar juntos. En el archivo `docker-compose.yml` describimos todos los nodos (los *servicios*) y los levantamos con un solo comando.

La red y el DNS interno. Compose crea una red privada para el sistema. Dentro de ella, cada nodo encuentra a los demás **por su nombre**: el nodo `app` no necesita saber la IP de `db`, porque Docker trae un DNS interno que traduce el nombre `db` a su dirección. Las IP cambian cada vez que se crean los contenedores; los nombres no.

localhost no es lo que parece. Para cada contenedor, `localhost` significa "yo mismo". Por eso, desde `app`, la base de datos no está en `localhost` sino en `db`: es otra máquina, aunque todo corra en el mismo computador.

Puertos. Un puerto es la puerta de entrada de un programa dentro de una máquina. PostgreSQL escucha en el 5432 y nuestro servidor web en el 8000. Con `ports: "8000:8000"` abrimos una puerta desde nuestro Windows hacia el contenedor `app`, para que el navegador pueda entrar. La base de datos no tiene esa línea a propósito: solo se puede llegar a ella desde dentro de la red del sistema.

El archivo .env. Laravel lee su configuración de este archivo: a qué base de datos conectarse, dónde guardar las sesiones, etc. Dos reglas que conviene recordar: una línea que empieza con `#` es un comentario y se ignora, y si una variable aparece dos veces, gana la última.

Código de construcción

Paso 1: un Laravel vacío. Sin instalar PHP en Windows, usamos un contenedor desechable con Composer para crear el proyecto:

```
docker run --rm -v "${PWD}:/app" composer create-project laravel/laravel subastas-clase
```

Funcionamiento en el navegador con un solo proceso y una base de datos SQLite (un simple archivo). Eso aún **no es** un sistema distribuido: todo vive en un solo lugar.

Paso 2: la receta de la máquina (Dockerfile). Se inicia con dos líneas y fuimos agregando solo lo que hacía falta. Con `php -m` comprobamos que la máquina no sabía hablar con PostgreSQL; agregamos la extensión y volvimos a comprobar. Más tarde un error nos pidió la extensión `intl`, y también la agregamos.

```
FROM php:8.4-cli
```

```
RUN apt-get update && apt-get install -y libpq-dev libicu-dev && docker-php-ext-install pdo_pgsql intl
```

```
WORKDIR /var/www/html
```



Paso 3: los nodos del sistema (docker-compose.yml). Primero solo db, una imagen ya hecha de PostgreSQL. Después app, construida con nuestro Dockerfile.

```
name: subastas-andres

services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: subastas
      POSTGRES_USER: subastas
      POSTGRES_PASSWORD: secret

  app:
    build: .
    volumes:
      - ./var/www/html
    command: php artisan serve --host=0.0.0.0 --port=8000
    ports:
      - "8000:8000"
    depends_on:
      - db
```

Comprobar que app encuentra a db por su nombre con `docker compose exec app getent hosts db`, que respondió una IP como `172.19.0.2`.

Paso 4: conectar Laravel con el nodo db (.env). Cambiamos SQLite por PostgreSQL. Los valores deben coincidir con los que db recibió en el compose:

```
DB_CONNECTION=pgsql
DB_HOST=db
DB_PORT=5432
DB_DATABASE=subastas
DB_USERNAME=subastas
DB_PASSWORD=secret
```

```
SESSION_DRIVER=file
CACHE_STORE=file
```

Con `php artisan migrate` creamos las tablas, esta vez en otro nodo y a través de la red. Con `php artisan db:show` confirmamos la conexión.

Paso 5: la página que revisa los nodos (routes/web.php). La ruta `/` responde quién la atiende y si la base de datos contesta:

Errores que nos pasaron y cómo los resolvimos

Equivocarse es parte de aprender, y estos errores son muy comunes. Si te aparece alguno, aquí está la solución.

Lo que vimos	Qué significaba	Cómo se resolvió
--------------	-----------------	------------------



Lo que vimos	Qué significaba	Cómo se resolvió
<code>cd : No se encuentra la ruta</code>	Usamos una ruta relativa desde la carpeta equivocada	Mirar la línea <code>PS C:\...></code> , que dice dónde estamos, o usar la ruta completa
<code>'docker buildx build' requires 1 argument</code>	Faltaba el punto final en <code>docker build -t prueba .</code>	El punto significa "esta carpeta": ahí está el Dockerfile
<code>no configuration file provided: not found</code>	El <code>docker-compose.yml</code> se creó dentro de la carpeta <code>app</code>	Moverlo a la raíz del proyecto. En VS Code, New File crea el archivo dentro de la carpeta seleccionada
El sistema se llamó distinto a lo que decía el editor	Docker lee lo guardado en disco, no lo que muestra el editor	Guardar (Ctrl + S) y revisar que se edita el archivo correcto
Un contenedor quedó encendido "huérfano"	Cambiamos el <code>name</code> : del <code>compose</code> sin apagar antes el sistema	Apagar primero con <code>docker compose down</code> , luego cambiar el nombre
The "intl" PHP extension is required	A la máquina le faltaba una extensión de PHP	Agregarla al Dockerfile y reconstruir con <code>--build</code>
Database file at path [subastas] does not exist	La línea <code>DB_CONNECTION</code> quedó comentada con <code>#</code> y Laravel volvió a SQLite	Quitar el <code>#</code>
La página se rompió entera con db apagado	<code>SESSION_DRIVER</code> estaba dos veces y ganaba <code>database</code>	Dejar una sola línea, con el valor <code>file</code>
Se encendió solo el proyecto de referencia	Sus servicios tienen <code>restart: unless-stopped</code> y arrancan al iniciar Docker	Apagarlo con <code>docker compose -p subastas down</code>

Un consejo para leer errores: no te asustes por la cantidad de texto. Busca primero las palabras **ERROR**, **FATAL** o **Exception**, y en las páginas de error de Laravel baja hasta **Previous exception**, donde suele estar la causa real. Los mensajes **LOG**, **warning** o **hint** son informativos.

Glosario de comandos

Todos se escriben en PowerShell, dentro de la carpeta del proyecto, salvo los que indican otra cosa. Los comandos `docker compose ...` buscan el archivo `docker-compose.yml` en la carpeta donde estás.

Moverse entre carpetas (PowerShell)



Comando	Qué hace
<code>cd C:\cursos\subastas</code>	Entra a una carpeta. Con ruta completa funciona estés donde estés
<code>dir</code>	Lista los archivos de la carpeta actual
<code>dir docker*</code>	Lista solo los archivos cuyo nombre empieza con "docker"
<code>Move-Item app\docker-compose.yml .</code>	Mueve un archivo a la carpeta actual (el punto es "aquí")

Docker: imágenes y contenedores sueltos

Comando	Qué hace
<code>docker run hello-world</code>	Crea un contenedor de prueba a partir de la imagen hello-world
<code>docker run --rm ...</code>	--rm borra el contenedor al terminar, para no dejar basura
<code>docker run -v "\${PWD}:/app" ...</code>	-v conecta tu carpeta actual con la carpeta /app del contenedor
<code>docker run -p 8000:8000 ...</code>	-p abre una puerta: el puerto 8000 de Windows lleva al 8000 del contenedor
<code>docker run -it ...</code>	Modo interactivo: se detiene con Ctrl + C
<code>docker build -t prueba .</code>	Construye una imagen con el Dockerfile de esta carpeta y la llama prueba
<code>docker run --rm prueba php -m</code>	Lista las extensiones de PHP que trae la imagen prueba
<code>docker ps</code>	Lista todos los contenedores encendidos, de cualquier proyecto
<code>docker ps -a</code>	Igual, incluidos los detenidos
<code>docker container prune</code>	Borra los contenedores detenidos (no toca imágenes, volúmenes ni archivos)

Docker Compose: el sistema completo

Comando	Qué hace
---------	----------



Comando	Qué hace
<code>docker compose up -d</code>	Crea y enciende todos los nodos del sistema. -d los deja corriendo en segundo plano
<code>docker compose up -d --build</code>	Igual, pero reconstruye la imagen porque el Dockerfile cambió
<code>docker compose ps</code>	Lista los nodos de este sistema y su estado
<code>docker compose logs db</code>	Muestra lo que el nodo db ha escrito desde que arrancó
<code>docker compose stop db</code>	Apaga solo el nodo db, sin borrarlo
<code>docker compose start db</code>	Vuelve a encender el nodo db
<code>docker compose restart app</code>	Apaga y enciende app, por ejemplo para que lea un <code>.env</code> nuevo
<code>docker compose down</code>	Apaga el sistema y borra sus contenedores (los archivos del proyecto no se tocan)
<code>docker compose -p subastas down</code>	Lo mismo, pero para el proyecto llamado subastas, sin importar la carpeta
<code>docker compose exec app <comando></code>	Ejecuta un comando dentro del nodo app

Dentro del nodo app (con `docker compose exec app ...`)

Comando	Qué hace
<code>getent hosts db</code>	Pregunta al DNS interno qué IP tiene el nombre db
<code>php artisan migrate</code>	Crea las tablas de Laravel en la base de datos
<code>php artisan db:show</code>	Muestra a qué base de datos está conectado Laravel y sus tablas